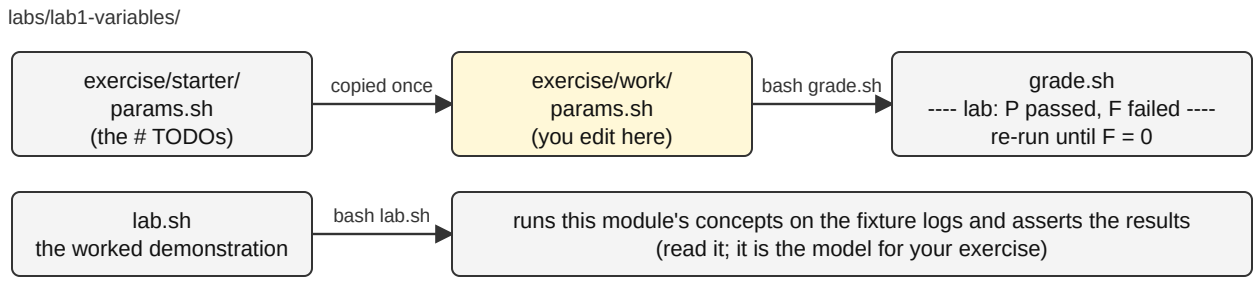
# Lab 1 — Variables, Quoting and Parameter Expansion: Three Helpers

## Topology




## Environment Table

All paths are relative to `labs/lab1-variables/exercise/`.

| File | What it is | You edit it? |
|---|---|---|
| `TASK.md` | The short statement of the exercise | No |
| `starter/params.sh` | Three functions that return `FIXME`, each with a `# TODO` | No — the grader copies it |
| `work/params.sh` | **Your** copy (created by the first `grade.sh` run) | **Yes** |
| `grade.sh` | The grader: 7 checks. It *sources* your file and calls the functions | No |
| `work/REFLECTION.md` | Your two reflection answers | **Yes** (you create it) |

## Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `stem` — turn a log path into a clean name with parameter expansion.
- **Part 3:** Build `report_name` — compose a file name from `stem` and a date.
- **Part 4:** Build `parse_kv` — read one `key = value` line into two variables, trimmed.
- **Part 5:** Verify with the grader; write the two reflection answers.

## Background / Scenario

`logsleuth` names its report files after the log it analysed (`report-access-2026-10-16.txt`) and reads its settings from a small `key = value` configuration file. Both jobs are done with **parameter expansion** — bash's built-in way of trimming and reshaping a value without starting another program. In this lab you write the three helpers `logsleuth` really uses: `stem`, `report_name` and `parse_kv`.

The grader *sources* `params.sh` and calls the functions directly, so the file only has to **define** the functions. The functions must print their result (or, for `parse_kv`, set the two global variables `KEY` and

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

`VALUE`) exactly as the checks expect.

## Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M01, steps S01.4–S01.6.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

## Part 1: Prepare the exercise folder

### Step 1: Run the grader once.

```
cd labs/lab1-variables/exercise
bash grade.sh
```

**Expected:** `---- lab1: 0 passed, 7 failed ----`, every FAIL showing `expected [...], got [FIXME]`. The checks run top to bottom: three for `stem`, one for `report_name`, three for `parse_kv`. Fix one function at a time.

### Step 2: Open `work/params.sh`.

Each function has a `# TODO` comment quoting the move it needs. The file must not run anything at the top level — it is sourced.

## Part 2: Build `stem`

`stem PATH` turns a log path into a clean name: `/var/log/nginx/access.log` → `access`; `auth.log` → `auth`; `a/b/c/app` → `app` (nothing to drop).

### Step 1: Strip the directory, then the suffix.

Replace the body of `stem` with:

```
stem() {
  local path="$1" base
  base="${path##*/}"              # strip everything up to the last '/'
  printf '%s\n' "${base%.log}"    # strip a trailing '.log'
}
```

How it works: `${path##*/}` removes the **longest** match of `*/` from the front — the whole directory part — leaving the file name. `${base%.log}` removes the **shortest** match of `.log` from the end; a name without that suffix is left untouched.

**Note:** One `#` would remove the *shortest* leading match (`/`), leaving `var/log/nginx/access.log`. The doubled `##` is what makes it the basename.

### Step 2: Try it by hand.

```
source work/params.sh
stem /var/log/nginx/access.log; stem auth.log; stem a/b/c/app
```

**Expected:** `access`, `auth`, `app`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

## Part 3: Build `report_name`

`report_name PATH DATE` prints `report-<stem>-<date>.txt`.

### Step 1: Compose the name.

```
report_name() {
  local path="$1" date="$2"
  printf 'report-%s-%s.txt\n' "$(stem "$path")" "$date"
}
```

`$(stem "$path")` captures what `stem` prints and drops it into the format. Each value is a separate, quoted `printf` argument.

### Step 2: Try it.

```
report_name /var/log/nginx/access.log 2026-10-16
```

**Expected:** `report-access-2026-10-16.txt`.

## Part 4: Build `parse_kv`

`parse_kv "top_n = 25"` must set the global variables `KEY=top_n` and `VALUE=25` with surrounding whitespace removed — also for a line like `" brute_threshold=10 "`.

### Step 1: Split on `=`, then trim.

```
parse_kv() {
  local line="$1" k v
  IFS='=' read -r k v <<< "$line"
  read -r KEY   <<< "$k"          # a bare read trims surrounding whitespace
  read -r VALUE <<< "$v"
}
```

How it works: `IFS='=' read -r k v <<< "$line"` splits the line at the first `=` into `k` and `v` (the `IFS='='` applies only to that one command). A second, plain `read -r` into each global strips leading and trailing spaces and tabs for free, because `read` uses the default `IFS` to trim.

**Note:** `KEY` and `VALUE` are deliberately **not** declared `local` — the caller needs them after the function returns. A `local` would make the grader see nothing.

### Step 2: Try it.

```
parse_kv "top_n = 25"; echo "[$KEY] [$VALUE]"
parse_kv "  brute_threshold=10  "; echo "[$KEY] [$VALUE]"
```

**Expected:** `[top_n] [25]` and `[brute_threshold] [10]` — no spaces inside the brackets.

## Part 5: Verify and reflect

### Step 1: Run the grader until it is green.

```
bash grade.sh
```

**Expected:** `---- lab1: 7 passed, 0 failed ----`. The seven checks: `stem` on `/var/log/nginx/access.log`, `auth.log` and `a/b/c/app`; `report_name` → `report-access-2026-10-`

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

`16.txt`; `parse_kv` key; `parse_kv` value; `parse_kv` trims around = and the whole line.

| Symptom | Cause | Fix |
|---|---|---|
| `stem` returns `var/…` | A single `#` strips the shortest match | Use `##` |
| `stem a/b/c/app` returns something other than `app` | The suffix removal is not `%.log` | `${base%.log}` leaves a name without `.log` untouched |
| `parse_kv` key or value keeps spaces | The second `read` is missing | Print `[$k]` and `[$v]` to see the spaces, then add the trimming reads |
| `parse_kv` checks see empty values | `KEY`/`VALUE` were declared `local` | Remove the `local` |

### Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each):

**Question 1:** Why do `${1:-world}` and `${x:-10}` use a colon? What changes without it?

**Question 2:** In S01.2 an unquoted `$path` became two arguments. What decides *where* an unquoted value splits, and why is quoting — not changing `IFS` — the right fix?

## Hand-in and grading

Hand in `work/params.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism. Unlimited attempts.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.